Service Média

Version 1.4 — Final — Aligné Auth v2.1 · Users v1.0 · Notification v1.2

Historique des Révisions

Ver.	Date	Description	Auteur
1.0	2026-03-21	Création initiale du document	Équipe Ingénierie
1.2	2026-03-21	Post-audit : alignement platform_id UUID (Auth v2.1). Contrats inter-services. Stack Node.js/NestJS. Rate limiting. RGPD. Préfixe /api/v1. Health check. Corrections MLD. PlantUML annexe.	Équipe Architecture
1.3	2026-03-21	Ajout owner_user_id. Contrat Users ↔ Média (photo/KYC). Séquence RGPD orchestrée. Diagramme async corrigé. Autorisation hard delete précisée. Sécurité SSRF from-url. Déduplication retirée du MLD. Consommateurs futurs.	Équipe Architecture
1.4	2026-03-21	Version finale : avatar_media_id (pas avatar_url) dans contrat Users. Dépendance Notification marquée futur (pas MVP). Hard delete S2S-only. Coquilles version corrigées.	Équipe Architecture

Table des matières

Historique des Révisions **1**

1 Introduction **4**
1.1 Objet du Document . . . 4
1.2 Destinataires . . . 4
1.3 Périmètre du Service . . . 4

2 Contexte & Problématique **4**
2.1 Pourquoi Ce Service Existe . . . 4
2.2 Positionnement dans l'Architecture . . . 5

3 Vision & Objectifs **5**
3.1 Concept . . . 5
3.2 Objectifs MVP & Long Terme . . . 5
3.3 KPIs . . . 5

4 Parties Prenantes **5**
4.1 Services Consommateurs . . . 6

5 Besoins Fonctionnels **6**
5.1 Module Upload . . . 6
5.1.1 Upload Simple . . . 6
5.1.2 Upload Multiple (Batch) . . . 6
5.1.3 Upload depuis URL Externe . . . 6
5.1.4 Types de Fichiers Supportés . . . 6
5.2 Module Traitement (Sharp / FFmpeg) . . . 6
5.3 Module Accès & Distribution . . . 7
5.4 Module Gestion . . . 7
5.5 Module Métadonnées . . . 7
5.6 Module Configuration Plateforme . . . 7
5.7 Module Statistiques . . . 7
5.8 Module Logs d'Accès . . . 7

6 Besoins Non-Fonctionnels **8**

7 Architecture Technique **8**
7.1 Stack Technique . . . 8
7.2 Flux de Traitement Asynchrone . . . 9
7.3 Provider de Stockage Abstrait . . . 9

8 Modélisation **9**
8.1 Table `media_files` . . . 10
8.2 Table `media_variants` . . . 10
8.3 Table `media_metadata` . . . 10
8.4 Table `platform_media_configs` . . . 11
8.5 Table `media_access_logs` . . . 11

9 Contrats d'API **11**

9.1 Health Check . . . 11
9.1.1 GET /api/v1/media/health . . . 11
9.2 Upload . . . 11
9.2.1 POST /api/v1/media . . . 11
9.2.2 POST /api/v1/media/batch . . . 11
9.2.3 POST /api/v1/media/from-url . . . 12
9.3 Consultation . . . 12
9.3.1 GET /api/v1/media/{id} . . . 12
9.3.2 GET /api/v1/media/{id}/info . . . 12
9.3.3 GET /api/v1/media/{id}/thumbnails . . . 12
9.3.4 GET /api/v1/media/{id}/thumbnail/{size} . . . 12
9.3.5 GET /api/v1/media/{id}/resize . . . 12
9.3.6 GET /api/v1/media . . . 12
9.3.7 GET /api/v1/media/{id}/access-logs . . . 12
9.4 Gestion . . . 12
9.4.1 PUT /api/v1/media/{id}/metadata . . . 12
9.4.2 PUT /api/v1/media/{id}/visibility . . . 12
9.4.3 DELETE /api/v1/media/{id} . . . 12
9.4.4 DELETE /api/v1/media/{id}/permanent . . . 12
9.4.5 GET /api/v1/media/{id}/signed-url . . . 13
9.4.6 DELETE /api/v1/media/by-user/{userId} . . . 13
9.5 Configuration Plateforme . . . 13
9.5.1 GET /api/v1/platforms/{platformId}/media-config . . . 13
9.5.2 PUT /api/v1/platforms/{platformId}/media-config . . . 13
9.6 Statistiques . . . 13
9.6.1 GET /api/v1/media/stats . . . 13
9.6.2 GET /api/v1/media/stats/{platformId} . . . 13

10 Contrats Inter-Services **13**
10.1 Média → Auth (JWT utilisateur) . . . 13
10.2 Média → Auth (token S2S) . . . 13
10.3 Users → Média (Photo de profil & Documents KYC) . . . 14
10.4 Users → Média (Purge RGPD) . . . 14
10.5 Média → Notification (Alertes quota) . . . 14

11 Infrastructure **14**
11.1 Variables d'Environnement . . . 15
11.2 Docker Compose . . . 15

12 Recommandations & Évolutions **16**

Annexe A — Glossaire **17**

Annexe B — Diagramme de Classes (PlantUML) **18**

1 Introduction

1.1 Objet du Document

Ce document constitue le **Cahier des Charges Technique et Fonctionnel** du **Service Média** d'AG Technologies, version 1.4 finale. Il intègre les corrections d'alignement avec Auth v2.1, Users v1.0 et Notification v1.2.

> **Convention d'identité (v1.4)** :
>
> `platform_id` = **UUID** du registre Auth (`platforms.id`).
> `uploaded_by` = `users_auth.id` — l'acteur technique qui a effectué l'upload (extrait du JWT `sub`). `NULL` si upload S2S.
> `owner_user_id` = `users_auth.id` — le propriétaire métier du fichier (ex : l'utilisateur dont c'est la photo de profil ou le document KYC). Fourni en body par le service consommateur. `NULL` si non applicable.
> Média ne stocke aucune table locale de plateformes ni d'utilisateurs. Le mapping vers `users_profiles.id` est résolu par le Service Users.

1.2 Destinataires

Destinataire	Rôle
Équipe Backend	Développement et maintenance du service
Équipe Frontend	Intégration des APIs d'upload et d'affichage
Équipes consommatrices	AGT-Market, AGT-Bot, Service Users, Service Chat
Lead Technique / Architecte	Validation des choix techniques
Chef de Projet	Suivi et validation fonctionnelle

1.3 Périmètre du Service

> **Inclus :** upload, validation, stockage, traitement (compression, thumbnails, conversion), contrôle d'accès, distribution CDN, métadonnées, suppression, archivage, purge RGPD, logs d'accès.

> **Hors périmètre :** gestion des utilisateurs (Service Users), authentification (Service Auth), envoi de notifications (Service Notification), logique métier des plateformes.

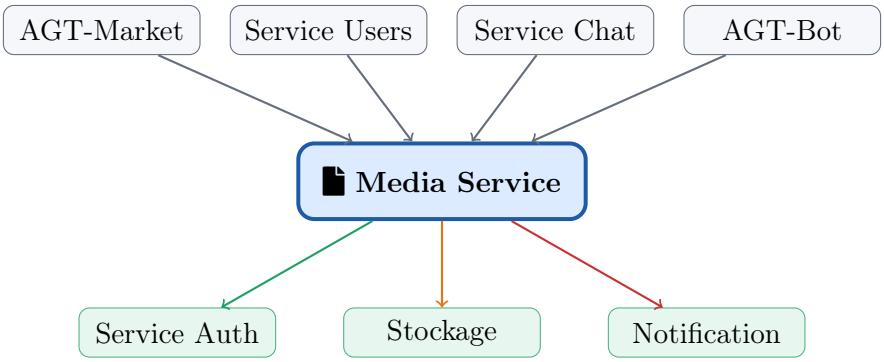
2 Contexte & Problématique

2.1 Pourquoi Ce Service Existe

Dans un écosystème multi-plateformes, la gestion des fichiers est un besoin transverse critique. Sans service centralisé : duplication de code, incohérence des traitements, risques de sécurité, coûts élevés.

2.2 Positionnement dans l'Architecture

Le Service Média est un **service de base**. Il dépend de **Auth** (validation tokens, dépendance MVP). La dépendance vers **Notification** (alertes quota) est **future** et n'est pas requise au MVP.



3 Vision & Objectifs

3.1 Concept

Service **générique** de fichiers. Il ne connaît pas le contexte métier. C'est la plateforme consommatrice qui donne du sens via les métadonnées.

3.2 Objectifs MVP & Long Terme

Phase	Objectifs
MVP	Upload/téléchargement ; validation ; stockage local VPS ; compression/-thumbnails (Sharp) ; accès public/privé ; URLs signées ; API REST `/api/v1` ; auth via Auth ; purge RGPD ; rate limiting
Long terme	Migration cloud (S3/GCS) ; CDN ; vidéo avancé (FFmpeg) ; quotas + alertes ; déduplication SHA-256 ; versioning ; monitoring avancé

3.3 KPIs

KPI	MVP	Long Terme
Upload 5 MB	< 3s	< 2s
Thumbnails	< 2s	< 1s
Resize (cache)	< 1s	< 500ms
Disponibilité	$\geq 99\%$	$\geq 99{,}5\%$
Taux erreur	$< 1\%$	$< 0{,}5\%$

4 Parties Prenantes

4.1 Services Consommateurs

Service	Usage
AGT-Market	Photos/vidéos produits, galeries, thumbnails
Service Users	Photos de profil, documents KYC (privés), purge RGPD
Service Chat	Pièces jointes (images, documents, audio)
AGT-Bot	Images réponse, vocaux WhatsApp, documents FAQ
Notification	Images dans emails et push
SALMA	Documents formation, ressources pédagogiques (migration prévue)
Futures plateformes	*Tout nouveau service de l'écosystème AGT peut consommer Média via les mêmes contrats (JWT ou S2S). Aucune modification de Média n'est requise.*

5 Besoins Fonctionnels

5.1 Module Upload

5.1.1 Upload Simple

- Upload via `multipart/form-data`
- Validation type MIME, extension, taille (selon config plateforme)
- Hash SHA-256 calculé à la réception
- `platform_id` extrait du JWT (`platform_id`) ou du token S2S (via introspection)
- `uploaded_by` = `sub` du JWT ; `NULL` pour uploads S2S
- `owner_user_id` (optionnel, en body) = propriétaire métier du fichier (`users_auth.id`). Utilisé quand un service ou un admin uploade pour le compte d'un utilisateur
- Déclenchement automatique du pipeline de traitement

5.1.2 Upload Multiple (Batch)

Upload de plusieurs fichiers en une requête. Réponse 207 Multi-Status avec statut par fichier.

5.1.3 Upload depuis URL Externe

Téléchargement depuis URL HTTP/HTTPS. Même pipeline que l'upload direct.

5.1.4 Types de Fichiers Supportés

Catégorie	Formats	Traitements
Images	JPEG, PNG, WebP, GIF	Compression, thumbnails, resize
Vidéos	MP4, WebM	Durée, thumbnail (FFmpeg)
Documents	PDF, DOCX, XLSX, CSV, TXT	Stockage brut
Audio	MP3, WAV, OGG	Durée, stockage brut

5.2 Module Traitement (Sharp / FFmpeg)

- **Compression** : qualité configurable par plateforme, original toujours conservé

— **Thumbnails** : tailles configurables (ex : 150×150, 300×300), stockées comme variantes
— **Resize à la volée** : endpoint dédié, résultat caché Redis

5.3 Module Accès & Distribution

Fichiers publics (URL directe), fichiers privés (token requis), URLs temporaires signées (durée configurable), CDN futur.

5.4 Module Gestion

Soft delete, mise à jour métadonnées, changement visibilité, listage paginé/filtrable, **purge RGPD** (suppression totale par utilisateur).

5.5 Module Métadonnées

Système (UUID, nom, MIME, taille, hash, dimensions, durée, dates) + personnalisées (clé-valeur libre par plateforme).

5.6 Module Configuration Plateforme

Types autorisés, tailles max par catégorie (JSONB), thumbnails, compression, quota. Approche **upsert**.

5.7 Module Statistiques

Espace global/plateforme/utilisateur, fichiers par MIME, fichiers populaires, bande passante.

5.8 Module Logs d'Accès

Chaque download et génération d'URL signée sont loggés. Consultation admin par fichier.

6 Besoins Non-Fonctionnels

ID	Exigence	Cible
NF-01	Upload 5 MB	< 3s
NF-02	Thumbnails après upload	< 2s
NF-03	Resize avec cache	< 1s
NF-04	Taille max défaut	50 MB (configurable)
NF-05	Auth obligatoire	JWT via `verify-token` ou S2S via `introspect`
NF-06	Rate limiting upload	30 req/min user, 100 req/min S2S
NF-07	Fichiers exécutables	Interdits par défaut
NF-08	Scalabilité	Horizontale, provider abstrait
NF-09	Disponibilité	$\geq$ 99,5%
NF-10	Health check	`GET /api/v1/media/health`
NF-11	Traitement async	BullMQ (Redis)

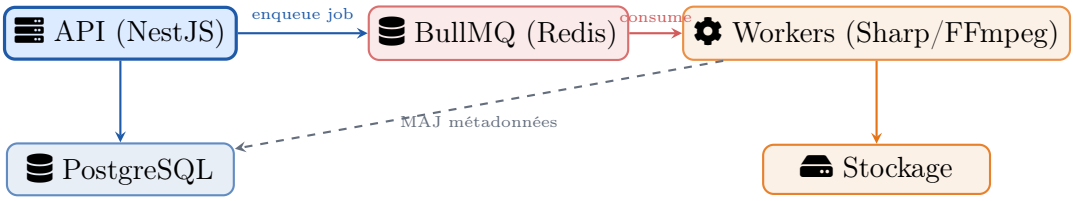
7 Architecture Technique

7.1 Stack Technique

Composant	Technologie	Justification
Runtime	Node.js 20 LTS	Performance I/O, streams natifs
Framework	NestJS	Modulaire, DI, TypeScript
Images	Sharp	Compression/resize haute perf
Vidéo/Audio	FFmpeg	Durée, thumbnail vidéo
BDD	PostgreSQL 16+	Métadonnées, configs, logs
ORM	TypeORM / Prisma	Mapping, migrations
Cache & Queue	Redis 7+ / BullMQ	Cache, rate limiting, async
Conteneur	Docker + Compose	Déploiement isolé

⚠ Auth, Users et Notification utilisent Django/DRF. Le choix Node.js/NestJS pour Média est délibéré : service I/O-intensif, Sharp 5–10× plus performant que Pillow. L'hétérogénéité de stack est acceptée en microservices.

7.2 Flux de Traitement Asynchrone



Flux (v1.4) : L'API enregistre le fichier brut et enqueue un job dans BullMQ. Les Workers consomment la queue et exécutent les traitements (compression, thumbnails, extraction durée). Les Workers mettent à jour les métadonnées et variantes dans PostgreSQL une fois le traitement terminé.

7.3 Provider de Stockage Abstrait

Listing 1 – Interface StorageProvider (TypeScript)

```
interface StorageProvider {
  upload(file: Buffer, key: string, opts?: UploadOptions): Promise<string
      >;
  download(key: string): Promise<Buffer>;
  delete(key: string): Promise<void>;
  getUrl(key: string, expiresIn?: number): Promise<string>;
  exists(key: string): Promise<boolean>;
}
```

Providers : `LocalStorageProvider` (MVP), `S3StorageProvider` (futur), `GCSStorageProvider` (futur).

Organisation : `/{platformUUID}/{année}/{mois}/{fileUUID}/`

8 Modélisation

8.1 Table `media_files`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
original_name	VARCHAR(255	NOT NULL	Nom original
mime_type	VARCHAR(100)	NOT NULL	Type MIME
size_bytes	BIGINT	NOT NULL	Taille en octets
sha256_hash	CHAR(64)	NOT NULL	Hash d'intégrité
storage_key	VARCHAR(500	NOT NULL	Clé stockage
visibility	VARCHAR(20)	NOT NULL	public / private
platform_id	UUID	NOT NULL	FK logique → Auth platforms.id
uploaded_by	UUID	NULL	FK logique → Auth users_auth.id
owner_user_id	UUID	NULL	Propriétaire métier. FK logique → Auth users_auth.id
width	INTEGER	NULL	Largeur (images/vidéos)
height	INTEGER	NULL	Hauteur
duration_ms	INTEGER	NULL	Durée (vidéo/audio)
deleted_at	TIMESTAMPTZ	NULL	Soft delete
created_at	TIMESTAMPT	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Mise à jour

8.2 Table `media_variants`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
media_id	UUID	FK	Fichier parent
variant_type	VARCHAR(20)	NOT NULL	thumbnail / compressed / resized
width	INTEGER	NULL	Largeur
height	INTEGER	NULL	Hauteur
storage_key	VARCHAR(500	NOT NULL	Clé stockage
size_bytes	BIGINT	NOT NULL	Taille
created_at	TIMESTAMPT	NOT NULL	Création

8.3 Table `media_metadata`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
media_id	UUID	FK	Fichier
key	VARCHAR(100)	NOT NULL	Clé
value	TEXT	NOT NULL	Valeur
created_at	TIMESTAMPTZ	NOT NULL	Création

8.4 Table `platform_media_configs`

Colonne	Type	Contrainte	Description
platform_id	UUID	PK	FK logique → Auth
allowed_types	JSONB	NOT NULL	Types MIME autorisés
max_size_bytes	JSONB	NOT NULL	Limites par catégorie
thumbnail_sizes	JSONB	NOT NULL	Tailles thumbnails
compression_quality	INTEGER	NULL	Qualité 1–100
storage_quota_byt	BIGINT	NULL	Quota optionnel
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPT	NOT NULL	Mise à jour

8.5 Table `media_access_logs`

Colonne	Type	Contrainte	Description
id	BIGSERIAL	PK	Auto-incrément
media_id	UUID	FK	Fichier accédé
accessed_by	UUID	NULL	Utilisateur
ip_address	INET	NULL	Adresse IP
action	VARCHAR(50)	NOT NULL	download / signed_url / view
created_at	TIMESTAMPT	NOT NULL	Horodatage

9 Contrats d'API

🔒 Tous les endpoints (sauf publics et signés) requièrent : `Authorization: Bearer <token>`
Validé via `GET /auth/verify-token` (JWT) ou `POST /auth/s2s/introspect` (S2S).
Préfixe : `/api/v1`

9.1 Health Check

9.1.1 `GET` /api/v1/media/health

Sans authentification. Retourne l'état du service et de ses dépendances.

9.2 Upload

9.2.1 `POST` /api/v1/media

Upload fichier unique. `platform_id` et `uploaded_by` extraits du token.
Body `multipart/form-data` : `file` (requis), `visibility` (défaut : public), `metadata` (JSON), `owner_user_id` (UUID, optionnel — propriétaire métier).
Si `visibility=public` : champ `url` renseigné, `signed_url` null.
Si `visibility=private` : champ `url` null, `signed_url` renseigné (TTL 1h).
Codes : 201, 400, 401, 413, 429, 500.

9.2.2 `POST` /api/v1/media/batch

Upload multiple. Réponse 207 Multi-Status.

9.2.3 POST /api/v1/media/from-url

Upload depuis URL externe. Body JSON : `url`, `visibility`, `metadata`, `owner_user_id` (optionnel).

> ⚠ **Sécurité SSRF (v1.4)** : règles obligatoires pour cet endpoint :
> — Schémas autorisés : `http`, `https` uniquement
> — Refus des IP privées, localhost, 127.0.0.0/8, 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.169.254 (metadata cloud)
> — Maximum 3 redirections HTTP
> — Timeout réseau : 10 secondes
> — Taille max téléchargée : selon `max_size_bytes` de la config plateforme
> — Résolution DNS vérifiée avant connexion (pas de DNS rebinding)

9.3 Consultation

9.3.1 GET /api/v1/media/{id}

Téléchargement binaire. Déclenche un log d'accès. Codes : 200, 401, 404.

9.3.2 GET /api/v1/media/{id}/info

Métadonnées uniquement (JSON).

9.3.3 GET /api/v1/media/{id}/thumbnails

Liste des variantes thumbnails.

9.3.4 GET /api/v1/media/{id}/thumbnail/{size}

Thumbnail spécifique (ex : 300x300).

9.3.5 GET /api/v1/media/{id}/resize

Resize à la volée. Query : `w`, `h`, `crop`. Cache Redis.

9.3.6 GET /api/v1/media

Listage paginé. Query : `platform_id`, `uploaded_by`, `owner_user_id`, `mime_type`, `visibility`, `from`, `to`, `page`, `limit`.

9.3.7 GET /api/v1/media/{id}/access-logs

Logs d'accès. Admin ou propriétaire. Paginé.

9.4 Gestion

9.4.1 PUT /api/v1/media/{id}/metadata

MAJ métadonnées personnalisées.

9.4.2 PUT /api/v1/media/{id}/visibility

Changement de visibilité.

9.4.3 DELETE /api/v1/media/{id}

Soft delete. 204 No Content.

9.4.4 DELETE /api/v1/media/{id}/permanent

Hard delete. **Autorisé uniquement via token S2S.** Média ne vérifie pas les rôles utilisateur — si un service consommateur souhaite offrir la suppression définitive à un

utilisateur final, c'est ce service qui valide les permissions puis appelle Média via son propre token S2S. Codes : 204, 401, 403, 404.

9.4.5 GET /api/v1/media/{id}/signed-url

URL temporaire signée. Query : `expires` (secondes, défaut 3600). Déclenche un log.

9.4.6 DELETE /api/v1/media/by-user/{userId}

RGPD Purge physique totale des fichiers d'un utilisateur. Token S2S uniquement. `userId` = `users_auth.id`. Supprime tous les fichiers où `uploaded_by = userId` OU `owner_user_id = userId`, ainsi que leurs variantes, métadonnées et logs d'accès.
Réponse 200 : `files_deleted`, `variants_deleted`, `storage_freed_bytes`.
Codes : 200, 401, 404, 500.

9.5 Configuration Plateforme

9.5.1 GET /api/v1/platforms/{platformId}/media-config

Config média d'une plateforme.

9.5.2 PUT /api/v1/platforms/{platformId}/media-config

Upsert : création ou mise à jour. S2S ou admin requis. Codes : 200/201, 400, 401.

9.6 Statistiques

9.6.1 GET /api/v1/media/stats

Stats globales.

9.6.2 GET /api/v1/media/stats/{platformId}

Stats par plateforme.

10 Contrats Inter-Services

10.1 Média → Auth (JWT utilisateur)

Param.	Valeur
Endpoint	`GET /auth/verify-token`
Auth	`Authorization: Bearer <access_token>`
Données	`user_id` (sub), `platform_id`
Cache	Redis TTL 30s

10.2 Média → Auth (token S2S)

Param.	Valeur
Endpoint	`POST /auth/s2s/introspect`
Données	`client_id` (= platform_id), `service_name`
Usage	Uploads S2S (AGT-Bot, Users, etc.)

10.3 Users → Média (Photo de profil & Documents KYC)

Param.	Valeur
Flux photo	Users appelle `POST /api/v1/media` (token S2S, `owner_user_id` = auth_user_id de l'utilisateur). Média retourne `media_id`. Users stocke ce `media_id` dans un champ dédié (ex : `avatar_media_id` de type UUID, distinct de `avatar_url`). L'URL publique est résolue à la lecture via `GET /api/v1/media/{id}/info`.
Flux KYC	Idem. Users stocke `media_id` dans `documents.media_id`. Média reçoit `visibility=private` et `metadata={"context":"kyc"}`.
Vérité métier	**Users** porte la logique métier (quel fichier est une photo de profil, quel fichier est un document KYC). **Média** reste générique et ne connaît pas le contexte métier.
Référencement	Users stocke les `media_id` dans ses propres tables. Média ne stocke aucune référence vers Users.

10.4 Users → Média (Purge RGPD)

Param.	Valeur
Endpoint	`DELETE /api/v1/media/by-user/{userId}`
Auth	Token S2S
Séquence	1. Users marque le profil `status=deletion_in_progress` 2. Users appelle Média purge (`userId` = `auth_user_id`) 3. Users appelle Auth purge 4. Users finalise son propre hard delete Le `userId` est conservé jusqu'à la fin du workflow.
Tolérance erreurs	Si Média échoue (500), Users conserve le statut `deletion_in_progress` et peut retenter. Aucune donnée n'est perdue. Un cron de rattrapage peut relancer les purges échouées.

10.5 Média → Notification (Alertes quota)

FUTUR — NON REQUIS AU MVP

Param.	Valeur
Endpoint	`POST /api/v1/notifications/send`
Déclencheur	Stockage plateforme $\geq$ 80% ou 100% du quota
Templates	`media_quota_warning` / `media_quota_exceeded`

11 Infrastructure

11.1 Variables d'Environnement

Variable	Description
STORAGE_PROVIDER	local | s3 | gcs
LOCAL_STORAGE_PATH	Chemin racine stockage
DATABASE_URL	PostgreSQL
REDIS_URL	Redis
AUTH_SERVICE_URL	URL Service Auth
NOTIFICATION_SERVICE_U	URL Service Notification (futur — alertes quota)
S2S_CLIENT_ID	Client ID pour tokens S2S
S2S_CLIENT_SECRET	Client secret
SIGNED_URL_SECRET	Clé HMAC URLs signées
SOFT_DELETE_RETENTION_	Délai avant purge
RATE_LIMIT_USER	30 req/min (défaut)
RATE_LIMIT_S2S	100 req/min (défaut)

11.2 Docker Compose

Listing 2 – docker-compose.yml — Media Service v1.4

```yaml
services:
  media-service:
    build: ./media-service
    ports:
      - "3003:3003"
    volumes:
      - media_storage:/data/media
    environment:
      - STORAGE_PROVIDER=local
      - LOCAL_STORAGE_PATH=/data/media
      - DATABASE_URL=postgresql://...
      - REDIS_URL=redis://redis:6379
      - AUTH_SERVICE_URL=http://auth-service:3001
      - NOTIFICATION_SERVICE_URL=http://notif-service:3004
    depends_on:
      - postgres
      - redis

  postgres:
    image: postgres:16-alpine
    volumes:
      - pg_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine

volumes:
  media_storage:
  pg_data:
```

12 Recommandations & Évolutions

- **Déduplication SHA-256 (futur v2)** : si hash identique pour même plateforme, retourner référence existante. Nécessitera un champ `ref_count` dans `media_files` — non modélisé en v1.4 pour garder le MLD simple au MVP
- **Migration Local → Cloud** : double écriture, migration async, bascule, vérification intégrité
- **Quotas + alertes** : 80% warning, 100% blocage (429), via Service Notification
- **Optimisation Web** : conversion WebP/AVIF auto (header Accept), compression progressive JPEG
- **Soft vs Hard delete** : soft = utilisateur, hard auto = cron après X jours, hard immédiat = admin, purge RGPD = S2S Users

Annexe A — Glossaire

Terme	Définition
CDN	Réseau de distribution de contenu
Soft Delete	Suppression logique, données conservées
Hard Delete	Suppression physique définitive
Signed URL	URL temporaire avec signature cryptographique
SHA-256	Hachage cryptographique pour intégrité
S2S	Service-to-Service (inter-microservices)
BullMQ	File d'attente Node.js basée sur Redis
Sharp	Librairie Node.js haute performance pour images

Annexe B — Diagramme de Classes (PlantUML)

Le code ci-dessous peut être rendu sur `plantuml.com` ou tout outil compatible.

Listing 3 – PlantUML — Media Service v1.4

```
@startuml Media Service - Class Diagram v1.4

skinparam classAttributeIconSize 0
skinparam linetype ortho

entity "media_files" as media_files {
  * id : UUID <<PK>>
  --
  * original_name : VARCHAR(255)
  * mime_type : VARCHAR(100)
  * size_bytes : BIGINT
  * sha256_hash : CHAR(64)
  * storage_key : VARCHAR(500)
  * visibility : VARCHAR(20)
  * platform_id : UUID <<FK logique -> Auth platforms.id>>
  uploaded_by : UUID <<FK logique -> Auth users_auth.id>>
  owner_user_id : UUID <<FK logique -> Auth users_auth.id>>
  width : INTEGER
  height : INTEGER
  duration_ms : INTEGER
  deleted_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
}

entity "media_variants" as media_variants {
  * id : UUID <<PK>>
  --
  * media_id : UUID <<FK>>
  * variant_type : VARCHAR(20)
  width : INTEGER
  height : INTEGER
  * storage_key : VARCHAR(500)
  * size_bytes : BIGINT
  * created_at : TIMESTAMPTZ
}

entity "media_metadata" as media_metadata {
  * id : UUID <<PK>>
  --
  * media_id : UUID <<FK>>
  * key : VARCHAR(100)
  * value : TEXT
  * created_at : TIMESTAMPTZ
}

entity "platform_media_configs" as platform_configs {
  * platform_id : UUID <<PK, FK logique -> Auth>>
  --
  * allowed_types : JSONB
  * max_size_bytes : JSONB
  * thumbnail_sizes : JSONB
  compression_quality : INTEGER
  storage_quota_bytes : BIGINT
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
}

entity "media_access_logs" as access_logs {
  * id : BIGSERIAL <<PK>>
  --
  * media_id : UUID <<FK>>
  accessed_by : UUID
  ip_address : INET
  * action : VARCHAR(50)
  * created_at : TIMESTAMPTZ
}
```

```
68
69 class "<<interface>>\nStorageProvider" as SP {
70   +upload(file, key) : String
71   +download(key) : Buffer
72   +delete(key) : void
73   +getUrl(key, ttl) : String
74   +exists(key) : Boolean
75 }
76
77 class "LocalStorageProvider" as LSP {
78   +base_path : String
79 }
80
81 class "S3StorageProvider" as S3P {
82   +bucket_name : String
83   +region : String
84 }
85
86 media_files ||--o{ media_variants : "has variants"
87 media_files ||--o{ media_metadata : "has metadata"
88 media_files ||--o{ access_logs : "has logs"
89 platform_configs ||--o{ media_files : "configures"
90
91 SP <|.. LSP : implements
92 SP <|.. S3P : implements
93 media_files ..> SP : "uses"
94
95 @enduml
```

AG Technologies — Document confidentiel à usage interne exclusivement.
Media Service CDC v1.4 — Mars 2026